

Aula 4

Organização de projeto e versionamento com Git

Desenvolvimento de Sistemas | Java Desktop com Eclipse

Produto da aula: projeto Java organizado, versionado e publicado em repositório GitHub.

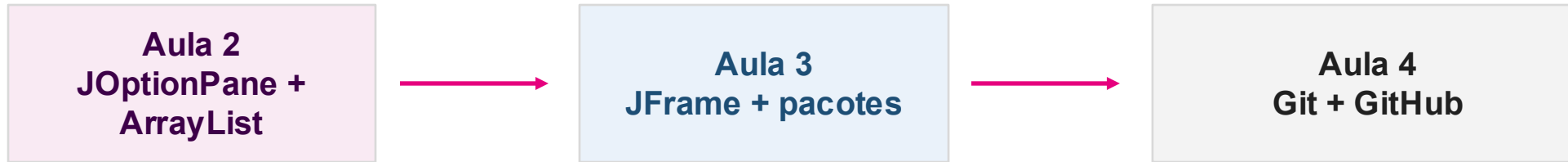
CEUB

EDUCAÇÃO SUPERIOR

ceub.br

De onde partimos?

A Aula 3 transformou o cadastro em tela Swing e separou responsabilidades.



- O código já tem model, service, view e app.
- Agora vamos transformar a pasta do projeto em histórico de versões.
- Cada alteração importante precisa ser pequena, testada e explicável.

Objetivos da aula

Ao final, o aluno deve conseguir organizar e versionar um projeto Java no Eclipse.

1
Estrutura

2
Git local

3
Commits

4
GitHub

5
Entrega

Pergunta guia: como saber o que mudou no projeto e quem conseguiu explicar essa mudança?

Por que organizar o projeto?

Um sistema cresce melhor quando cada classe tem um lugar e cada alteração tem uma razão.

**Encontrar
classes**

**Separar
responsabilidades**

**Reduzir
erros**

**Preparar
projeto final**

- Organização não é estética: é manutenção.
- Quando a turma voltar ao código uma semana depois, a estrutura precisa ajudar.
- Git registra a evolução; pacotes deixam a evolução compreensível.

Estrutura esperada

O projeto da Aula 4 parte da organização criada na Aula 3.

```
1 Aula04CadastroAlunos
2 |   src
3 |   |   br/edu/ceub/ds/aula04
4 |   |   |   model / Aluno.java
5 |   |   |   service / CadastroAlunos.java
6 |   |   |   view / TelaCadastroAluno.java
7 |   |   |   app / Principal.java
8 |   README.md
9 |   .gitignore
```

model: dados do domínio

service: regras e operações

view: tela Swing

app: ponto de entrada

Regra prática: se uma classe faz tudo, o projeto fica difícil de explicar e versionar.

Modelo mental do Git

Git não é só “salvar”: é registrar passos compreensíveis da construção.



```
1 git status
2 git add .
3 git commit -m "organiza estrutura inicial"
4 git push
```

- Antes de cada commit: compilar, executar e entender a mudança.
- Depois do push: conferir se o repositório ficou legível no GitHub.

Commits pequenos e explicáveis

Um commit deve contar uma etapa da construção, não misturar tudo.

Commit ruim

“alterações”
“trabalho”
“final”

Mistura arquivos, não explica intenção e dificulta correção.

Commit bom

“cria estrutura de pacotes”
“adiciona tela Swing de cadastro”
“valida semestre antes de cadastrar”

Mostra intenção, etapa e responsabilidade.

Mensagem útil: verbo no presente + mudança objetiva.

README e .gitignore

O repositório deve ser compreensível para outra pessoa abrir e executar.

```
1 # Cadastro de Alunos
2
3 Projeto Java Desktop desenvolvido em aula.
4
5 ## Como executar
6 1. Abrir no Eclipse
7 2. Executar
br.edu.ceub.ds.aula04.app.Principal
8
9 ## Organização
10 - model: entidades
11 - service: regras
12 - view: interface
13 - app: inicialização
```

```
1 bin/
2 target/
3 out/
4 *.class
5 .DS_Store
6 Thumbs.db
```

- README explica o projeto.
- .gitignore evita arquivos gerados.
- Código-fonte deve aparecer em src.

Roteiro da prática guiada

A professora demonstra; depois a turma executa e registra evidências.

1
Abrir projeto da Aula 3

2
Conferir pacotes

3
Criar README e .gitignore

4
Inicializar Git

5
Fazer commit inicial

6
Alterar pequena função

7
Fazer segundo commit

8
Publicar no GitHub

Prática guiada no Eclipse

O foco é aprender o fluxo: alterar, testar, versionar, explicar.

1. Team > Share Project
criar repositório local

2. Git Staging
selecionar arquivos corretos

3. Commit
escrever mensagem clara

4. Push
GitHub

```
1 public int contarAlunos() {  
2     return alunos.size();  
3 }  
4  
5 private void mostrarTotal() {  
6     areaResultado.setText("Total de alunos: " +  
cadastro.contarAlunos());  
7 }
```

- A alteração prática será pequena: botão Total de alunos.
- O commit deve registrar exatamente essa melhoria.
- O link final precisa abrir o repositório com README visível.

Atividade pontuada

Em dupla, entregar um repositório organizado e explicar o histórico.

A
Botão Total

B
**Validação de
semestre**

C
Busca por curso

D
**Bloqueio de
duplicado**

- Repositório GitHub com README e .gitignore.
- Pelo menos três commits: estrutura, documentação e melhoria.
- Funcionalidade funcionando no Eclipse.
- Explicação do caminho: arquivo alterado, regra, teste e commit.

Checklist de entrega

Antes de chamar a professora, confira os pontos essenciais.

Compila sem erro

Abre a janela

README existe

Gitignore existe

3 commits claros

Link abre no GitHub

**Funcionalidade
testada**

Dupla explica

Repositório sem explicação não vale como evidência completa de aprendizagem.

Erros comuns

A aula também ensina a interpretar problemas de versionamento.

Sintoma	Causa provável	Como resolver
Arquivos bin/ no GitHub	Commit incluiu gerados	Ajustar .gitignore e remover do staging
Nada aparece para commit	Projeto não foi compartilhado	Usar Team > Share Project
Push falha	Remote ou login ausente	Conferir URL/token no GitHub
Código não compila após clone	Faltou src ou pacote mudou	Conferir estrutura e imports

Fechamento

Da próxima vez: arquitetura de software, camadas e responsabilidades

Para revisar: qual commit melhor explica o que mudou no seu projeto?

model | service | view | app | README | commits